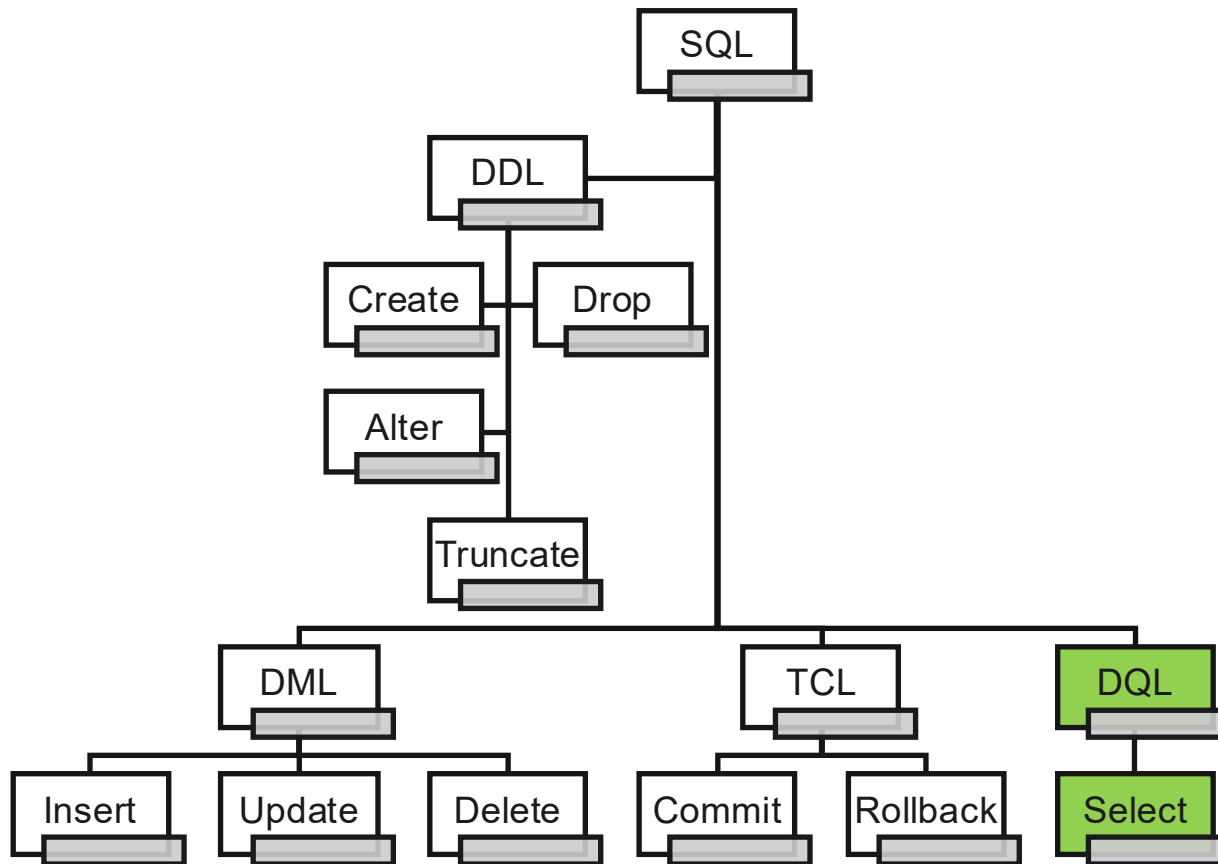


SQL: DQL

Vanessa Braganholo

vanessa@ic.uff.br

Aula de Hoje



Esquema usado nos exemplos

```
CREATE TABLE DEPARTAMENTO (
  COD_DEPTO INT NOT NULL,
  NOME VARCHAR(15) NOT NULL,
  CPF_GERENTE VARCHAR(11),
  DATA_INICIO_GERENTE DATE,
  PRIMARY KEY (COD_DEPTO),
  UNIQUE (NOME),
  FOREIGN KEY (CPF_GERENTE) REFERENCES EMPREGADO(CPF));
```

```
CREATE TABLE EMPREGADO (
  NOME VARCHAR(15) NOT NULL,
  INICIALM CHAR,
  SOBRENOME VARCHAR(15) NOT NULL,
  CPF VARCHAR(11) NOT NULL,
  DNASCIMENTO DATE,
  ENDERECO VARCHAR(30),
  SEXO CHAR,
  SALARIO FLOAT,
  CPF_SUPERVISOR VARCHAR(11),
  COD_DEPTO INT NOT NULL,
  PRIMARY KEY (CPF),
  FOREIGN KEY (CPF_SUPERVISOR) REFERENCES EMPREGADO (CPF),
  FOREIGN KEY (COD_DEPTO) REFERENCES DEPARTAMENTO (COD_DEPTO));
```

```
CREATE TABLE DEPTO_LOCALIZACOES (
  COD_DEPTO INT NOT NULL,
  LOCAL VARCHAR(15) NOT NULL,
  PRIMARY KEY (COD_DEPTO),
  FOREIGN KEY (COD_DEPTO) REFERENCES DEPARTAMENTO (COD_DEPTO));
```

```
CREATE TABLE PROJETO (
  COD_PROJ INT NOT NULL,
  NOME VARCHAR(30) NOT NULL,
  LOCAL VARCHAR(15),
  COD_DEPTO INT NOT NULL,
  PRIMARY KEY (COD_PROJ),
  UNIQUE (NOME),
  FOREIGN KEY (COD_DEPTO) REFERENCES DEPARTAMENTO (COD_DEPTO));
```

```
CREATE TABLE TRABALHA_EM (
  CPF VARCHAR(11) NOT NULL,
  COD_PROJ INT NOT NULL,
  HORAS FLOAT NOT NULL,
  PRIMARY KEY (CPF, COD_PROJ),
  FOREIGN KEY (CPF) REFERENCES EMPREGADO (CPF),
  FOREIGN KEY (COD_PROJ) REFERENCES PROJETO (COD_PROJ));
```

```
CREATE TABLE DEPENDENTE (
  EMP_CPF VARCHAR(11) NOT NULL,
  NOME VARCHAR(15) NOT NULL,
  SOBRENOME VARCHAR(15) NOT NULL,
  SEXO CHAR,
  DNASCIMENTO DATE,
  PARENTESCO VARCHAR(8),
  PRIMARY KEY (EMP_CPF, NOME),
  FOREIGN KEY (EMP_CPF) REFERENCES EMPREGADO(CPF));
```

EMPREGADO

Í	NOME	INICIA...	SOBR...	CPF	DNAS...	ENDE...	SEXO	SALA...	CPF_S...	COD_DEPTO
	John	B	Smith	12345...	1965-01...	731 Fon...	M	30000	33344...	5
	Franklin	T	Wong	33344...	1955-12...	638 Vos...	M	40000	88866...	5
	Joyce	A	English	45345...	1972-07...	5631 Ri...	F	25000	33344...	5
	Ramesh	K	Narayan	66688...	1962-09...	975 Fire...	M	38000	33344...	5
	James	E	Borg	88866...	1937-11...	450 Sto...	M	55000	NULL	1
	Jennifer	S	Wallace	98765...	1941-06...	291 Ber...	F	43000	88866...	4
	Ahmad	V	Jabbar	98798...	1969-03...	980 Dall...	M	25000	98765...	4
	Alicia	J	Zelaya	99988...	1968-01...	3321 C...	F	25000	98765...	4

DEPARTAMENTO

COD_DEPTO	NOME	CPF_GERENTE	DATA_INICIO_GERENTE
1	Sede administrativa	88866555500	1981-06-19
4	Administracao	98765432100	1995-01-01
5	Pesquisa	33344555500	1988-05-22

DEPTO_LOCALIZACOES

COD_DEPTO	LOCAL
1	Houston
4	Staffor
5	Bellaire

PROJETO

COD_PROJ	NOME	LOCAL	COD_DEPTO
1	ProdutoX	Bellaire	5
2	ProdutoY	Sugarland	5
3	ProdutoZ	Houston	5
10	Automatizacao	Stafford	4
20	Reorganizacao	Houston	1
30	Novos Beneficios	Stafford	4

TRABALHA_EM

CPF	COD_PROJ	HORAS
12345678900	1	32.5
12345678900	2	7.5
33344555500	2	10
33344555500	3	10
33344555500	10	10
33344555500	20	10
45345345300	1	20
45345345300	2	20
66688444400	3	40
88866555500	20	40
98765432100	20	15
98765432100	30	20
98798798700	10	35
98798798700	30	5
99988777700	10	10
99988777700	30	30

DEPENDENTE

EMP_CPF	NOME	SOBRENOME	SEXO	DNASCIMENTO	PARENTESCO
12345678900	Alice	Smith	F	1988-12-30	FILHA
12345678900	Elizabeth	Smith	F	1967-05-05	CONJUGE
12345678900	Michael	Smith	M	1988-01-04	FILHO
33344555500	Alice	Braga	F	1986-04-05	FILHA
33344555500	Joy	Silva	F	1958-05-03	CONJUGE
33344555500	Theodore	Silva	M	1983-10-25	FILHO
98765432100	Abner	Soares	M	1942-02-28	CONJUGE

Consultas mais “Complexas”

Recursos adicionais que permitem que os usuários especifiquem consultas mais complexas:

- Consultas aninhadas
- Tabelas de junção
- Junções externas
- Operações de conjuntos
- Funções agregadas
- Agrupamento

Consultas Aninhadas

Consulta Aninhada

Consultas aninhadas

- São blocos SELECT-FROM-WHERE completos dentro da cláusula WHERE de outra consulta (chamada de consulta externa)

Operador de comparação IN

- Compara um valor **v** com um conjunto (ou multiconjunto) de valores **V**
- Avalia como TRUE se **v** for um dos elementos em **V**

Consulta Aninhada

Faça uma lista de todos os nomes de projeto do departamento de Pesquisa

```
SELECT NOME
FROM PROJETO
WHERE COD_DEPTO IN (SELECT COD_DEPTO
                     FROM DEPARTAMENTO
                     WHERE NOME = 'Pesquisa')
```

Consulta Aninhada

Faça uma lista de todos os nomes de projeto do departamento de Pesquisa

Consulta Aninhada
ou Subconsulta

```
SELECT NOME
FROM PROJETO
WHERE COD_DEPTO IN (SELECT COD_DEPTO
                     FROM DEPARTAMENTO
                     WHERE NOME = 'Pesquisa')
```

Consulta Aninhada

Faça uma lista de todos os nomes de projeto do departamento de Pesquisa

```
SELECT NOME
```

```
FROM PROJETO
```

```
WHERE COD_DEPTO IN (SELECT COD_DEPTO
```

```
FROM DEPARTAMENTO
```

```
WHERE NOME = 'Pesquisa')
```

Consulta Externa

Consulta Aninhada – uso de tuplas

Permite o uso de tuplas de valores em comparações

- É necessário colocar os atributos envolvidos **entre parênteses**

Consulta Aninhada – uso de tuplas

Faça uma lista de CPFs de empregados que possuem dependentes do mesmo sexo que ele

```
SELECT CPF
FROM EMPREGADO
WHERE (CPF, SEXO) IN (SELECT EMP_CPF, SEXO
                      FROM DEPENDENTE) ;
```

Consulta Aninhada Correlacionada

Uma consulta aninhada é dita **correlacionada** quando ela usa atributos da tabela da consulta externa

Exemplo: Faça uma consulta que retorne os nomes e sobrenomes dos empregados que possuem dependentes que tenham o mesmo primeiro nome e o mesmo sexo do empregado.

```
SELECT NOME, SOBRENOME
FROM EMPREGADO AS e
WHERE CPF IN (SELECT EMP_CPF
              FROM DEPENDENTE AS d
              WHERE e.NOME = d.NOME
              AND e.SEXO = d.SEXO);
```

Consulta Aninhada Correlacionada

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Smith, seja como gerente do departamento onde o projeto é realizado ou como trabalhador do projeto

```
SELECT DISTINCT COD_PROJ
FROM PROJETO AS p
WHERE COD_DEPTO IN (SELECT COD_DEPTO
                     FROM DEPARTAMENTO AS d, EMPREGADO AS e
                     WHERE p.COD_DEPTO = d.COD_DEPTO
                     AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Smith')
OR
COD_PROJ IN (SELECT COD_PROJ
              FROM TRABALHA_EM AS t, EMPREGADO AS e
              WHERE e.CPF = t.CPF AND SOBRENOME = 'Smith');
```

EXISTS

Função EXISTS

- Verifica se o resultado de uma consulta aninhada correlacionada é vazio ou não

EXISTS e NOT EXISTS

- Costumam ser usados em conjunto com uma consulta aninhada correlacionada

EXISTS

Faça uma consulta que retorne os nomes e sobrenomes dos empregados que possuem dependentes que tenham o mesmo primeiro nome e o mesmo sexo do empregado.

```
SELECT NOME, SOBRENOME
FROM EMPREGADO e
WHERE EXISTS (SELECT *
              FROM DEPENDENTE d
              WHERE e.CPF = d.EMP_CPF
              AND e.NOME = d.NOME
              AND e.SEXO = d.SEXO) ;
```

NOT EXISTS

Faça uma consulta que retorne os nomes e sobrenomes dos empregados que não possuem dependentes.

```
SELECT NOME, SOBRENOME
FROM EMPREGADO e
WHERE NOT EXISTS (SELECT *
                   FROM DEPENDENTE d
                   WHERE e.CPF = d.EMP_CPF);
```

Consulta Aninhada na Cláusula FROM

Faça uma consulta que retorna o CPF do empregado cuja soma das horas trabalhadas em projetos é a menor de todas

```
SELECT CPF
FROM TRABALHA_EM
GROUP BY CPF
HAVING SUM(HORAS) = (
    SELECT MIN(total_horas)
    FROM (SELECT CPF, SUM(HORAS) as total_horas
          FROM TRABALHA_EM GROUP BY CPF) t1);
```

Consulta Aninhada na Cláusula FROM

Faça uma consulta que retorna o CPF do empregado cuja soma das horas trabalhadas em projetos é a menor de todas

```
SELECT CPF
FROM TRABALHA_EM
GROUP BY CPF
HAVING SUM(HORAS) = (
    SELECT MIN(total_horas)
    FROM (SELECT CPF, SUM(HORAS) as total_horas
          FROM TRABALHA_EM GROUP BY CPF) t1);
```

Essa tabela não tem nome, pois é uma consulta. Por isso a necessidade de dar um nome a ela (t1, nesse caso)

Tabelas de Junção

Tabela de Junção

- Permite aos usuários especificar uma tabela resultante de uma operação de junção na cláusula FROM de uma consulta

A cláusula FROM na consulta a seguir contém uma única tabela de junção

```
SELECT e.NOME, SOBRENOME, ENDERECO  
FROM (EMPREGADO AS e JOIN DEPARTAMENTO AS d  
        ON e.COD_DEPTO = d.COD_DEPTO)  
WHERE d.NOME = 'Pesquisa';
```

Consulta com 2 Junções

```
SELECT COD_PROJ, SOBRENOME, ENDERECO, DNASCIMENTO
FROM PROJETO p JOIN DEPARTAMENTO d
      ON p.COD_DEPTO = d.COD_DEPTO
      JOIN EMPREGADO e ON e.CPF = d.CPF_GERENTE
WHERE p.LOCAL = 'Niterói';
```

Junções Externas

Tipos de Junção

INNER JOIN

LEFT OUTER JOIN

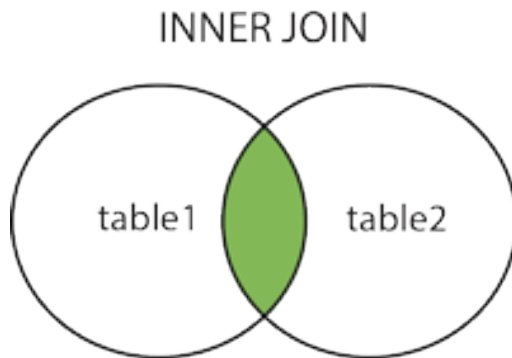
RIGHT OUTER JOIN

FULL OUTER JOIN

INNER JOIN

O tipo padrão de junção em uma tabela de junção

Tupla é incluída no resultado somente se uma tupla combinar na outra relação



Exemplo de INNER Join

```
SELECT e.NOME, SOBRENOME, ENDERECO
FROM (EMPREGADO AS e JOIN DEPARTAMENTO AS d
        ON e.COD_DEPTO = d.COD_DEPTO)
WHERE d.NOME = 'Pesquisa';
```

NOME	SOBRENOME	ENDERECO
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
John	Smith	731 Fondren, Houston, TX
Joyce	English	5631 Rice, Houston, TX

Relembrando o conteúdo das tabelas

DEPARTAMENTO

COD_DEPTO	NOME	CPF_GERENTE	DATA_INICIO_GERENTE
1	Sede administrativa	88866555500	1981-06-19
4	Administracao	98765432100	1995-01-01
5	Pesquisa	33344555500	1988-05-22

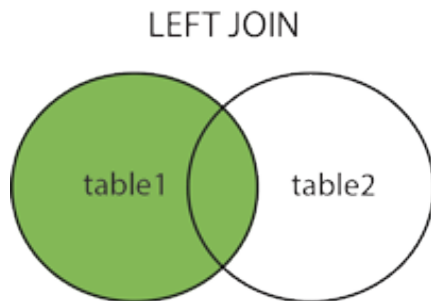
EMPREGADO

NOME	INICIA...	SOBR...	CPF	DNAS...	ENDE...	SEXO	SALA...	CPF_S...	COD_DEPTO
John	B	Smith	12345...	1965-01...	731 Fon...	M	30000	33344...	5
Franklin	T	Wong	33344...	1955-12...	638 Vos...	M	40000	88866...	5
Joyce	A	English	45345...	1972-07...	5631 Ri...	F	25000	33344...	5
Ramesh	K	Narayan	66688...	1962-09...	975 Fire...	M	38000	33344...	5
James	E	Borg	88866...	1937-11...	450 Sto...	M	55000	NULL	1
Jennifer	S	Wallace	98765...	1941-06...	291 Ber...	F	43000	88866...	4
Ahmad	V	Jabbar	98798...	1969-03...	980 Dall...	M	25000	98765...	4
Alicia	J	Zelaya	99988...	1968-01...	3321 C...	F	25000	98765...	4

LEFT OUTER JOIN

Toda tupla na tabela da esquerda precisa aparecer no resultado;

Se não existir uma tupla combinando na tabela da direita, tupla no resultado é preenchida com valores NULL para os atributos da tabela da direita



Exemplo de LEFT OUTER JOIN

Selecionar os empregados e seus supervisores. Empregados que não possuem supervisor devem mesmo assim aparecer na resposta

```
SELECT e.NOME AS NOME_EMPREGADO, s.NOME AS NOME_SUPERVISOR  
FROM EMPREGADO e LEFT OUTER JOIN EMPREGADO s  
           ON e.CPF_SUPERVISOR = s.CPF;
```

Exemplo de LEFT OUTER JOIN

```

SELECT e.NOME AS NOME_EMPREGADO, s.NOME AS NOME_SUPERVISOR
FROM EMPREGADO e LEFT OUTER JOIN EMPREGADO s
           ON e.CPF_SUPERVISOR = s.CPF;
  
```

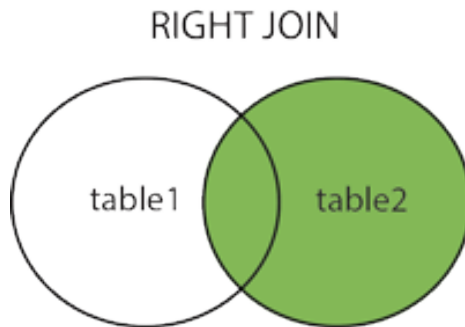
NOME_EMPREGADO	NOME_SUPERVISOR
James	NULL
Franklin	James
Ramesh	Franklin
John	Franklin
Jennifer	James
Ahmad	Jennifer
Alicia	Jennifer
Joyce	Franklin



RIGHT OUTER JOIN

Toda tupla na tabela da direita precisa aparecer no resultado

Se não existir uma tupla combinando na tabela da esquerda, tupla no resultado é preenchida com valores NULL para os atributos da tabela da esquerda



OBS: RIGHT OUTER JOIN só existe no SQLite a partir da versão 3.39 (junho de 2022)

- Confira a versão do seu SQLite com `SELECT sqlite_version();`
- Em versões antigas, é possível para simular com LEFT JOIN invertido

Exemplo de RIGHT OUTER JOIN

Selecionar os departamentos e os projetos de cada um.
Departamentos que não possuem projetos devem mesmo assim aparecer na resposta

```
SELECT p.NOME AS PROJETO, d.NOME AS DEPARTAMENTO  
FROM PROJETO p RIGHT OUTER JOIN DEPARTAMENTO d  
      ON p.COD_DEPTO = d.COD_DEPTO;
```

Relembrando o conteúdo das tabelas

DEPARTAMENTO

COD_DEPTO	NOME	CPF_GERENTE	DATA_INICIO_GERENTE
1	Sede administrativa	88866555500	1981-06-19
4	Administracao	98765432100	1995-01-01
5	Pesquisa	33344555500	1988-05-22

Não há departamento sem projeto, portanto essa consulta retorna o mesmo que o INNER JOIN retornaria.

Se adicionarmos um departamento novo, sem projeto, a resposta seria conforme o slide seguinte.

PROJETO

COD_PROJ	NOME	LOCAL	COD_DEPTO
1	ProdutoX	Bellaire	5
2	ProdutoY	Sugarland	5
3	ProdutoZ	Houston	5
10	Automatizacao	Stafford	4
20	Reorganizacao	Houston	1
30	Novos Beneficios	Stafford	4

Exemplo de RIGHT OUTER JOIN

```
SELECT p.NOME AS PROJETO, d.NOME AS DEPARTAMENTO
FROM PROJETO p RIGHT OUTER JOIN DEPARTAMENTO d
ON p.COD_DEPTO = d.COD_DEPTO;
```

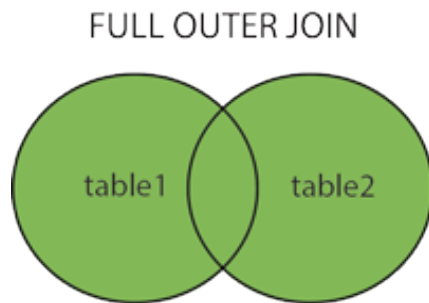
PROJETO	DEPARTAMENTO
ProdutoX	Pesquisa
ProdutoY	Pesquisa
ProdutoZ	Pesquisa
Automatizacao	Administracao
Reorganizacao	Sede administrativa
Novos Beneficios	Administracao
NULL	Novo



FULL OUTER JOIN

Toda tupla das duas tabelas precisa aparecer no resultado

Se não existir tupla combinando na tabela da direita ou da esquerda, tupla no resultado é preenchida com NULL no lado faltante



OBS: FULL OUTER JOIN só existe no SQLite a partir da versão 3.39 (junho de 2022)

- Confira a versão do seu SQLite com `SELECT sqlite_version();`
- Em versões antigas, é possível para simular com UNION

Exemplo de FULL OUTER JOIN

Selecionar os departamentos e seus gerentes. Departamentos que ainda não têm gerente e empregados que não gerenciam nenhum departamento devem aparecer na resposta

```
SELECT d.NOME AS DEPARTAMENTO, e.NOME AS GERENTE  
FROM DEPARTAMENTO d FULL OUTER JOIN EMPREGADO e  
    ON d.CPF_GERENTE = e.CPF;
```

Exemplo de FULL OUTER JOIN

```

SELECT d.NOME AS DEPARTAMENTO, e.NOME AS GERENTE
FROM DEPARTAMENTO d FULL OUTER JOIN EMPREGADO e
ON d.CPF_GERENTE = e.CPF;
  
```

DEPARTAMENTO	GERENTE
Sede administrativa	James
Administracao	Jennifer
Pesquisa	Franklin
Novo	NULL
NULL	Ramesh
NULL	John
NULL	Ahmad
NULL	Alicia
NULL	Joyce



Operações de Conjuntos

Recapitulando: Tabelas como Conjuntos

SQL não elimina automaticamente tuplas duplicadas nos resultados das consultas (resultado não é um conjunto)

Para garantir que o resultado contenha apenas tuplas distintas, usa-se a palavra-chave **DISTINCT** na cláusula **SELECT**

Recapitulando: Tabelas como Conjuntos

```
SELECT SALARIO  
FROM EMPREGADO;
```

SALARIO
55000
40000
38000
30000
43000
25000
25000
25000

```
SELECT DISTINCT SALARIO  
FROM EMPREGADO;
```

SALARIO
55000
40000
38000
30000
43000
25000

Tabelas como Conjuntos

Operações de conjunto (**duplicatas são eliminadas**):

- UNION
- EXCEPT (diferença)
- INTERSECT

Operações multi-conjuntos correspondentes (**duplicatas NÃO são eliminadas**):

- UNION ALL
- EXCEPT ALL (não existe no SQLite)
- INTERSECT ALL (não existe no SQLite)

Tabelas como Conjuntos

Em todos os casos, tabelas precisam ser **compatíveis para união**

- Colunas nas mesmas posições nas duas tabelas unidas precisam ter domínios compatíveis

UNION

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Wallace, seja como gerente do departamento onde o projeto é realizado ou como trabalhador do projeto

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

UNION

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

UNION

Domínio das
colunas é o
mesmo

Faça uma lista de todos os nomes de projetos e nomes de departamentos

```
SELECT p.NOME
FROM PROJETO p
UNION
SELECT d.NOME
FROM DEPARTAMENTO d
```

Nome
Administracao
Automatizacao
Novos Beneficios
Pesquisa
ProdutoX
ProdutoY
ProdutoZ
Reorganizacao
Sede administrativa

UNION

Domínio das
colunas é o
mesmo, mas
nome é diferente

Faça uma lista de todos os sobrenomes de empregados e nomes de departamentos

```
SELECT e.SOBRENOME
FROM EMPREGADO e
UNION
SELECT d.NOME
FROM DEPARTAMENTO d
```

SOBRENOME
Administracao
Borg
English
Jabbar
Narayan
Pesquisa
Sede administrativa
Smith
Wallace
Wong
Zelaya

UNION

Mesmos
nomes de
colunas

Faça uma lista de todos os sobrenomes de empregados e nomes de departamentos

```
SELECT e.SOBRENOME AS NOME
FROM EMPREGADO e
UNION
SELECT d.NOME
FROM DEPARTAMENTO d
```

⋮ NOME

Administracao

Borg

English

Jabbar

Narayan

Pesquisa

Sede administrativa

Smith

Wallace

Wong

Zelaya

UNION

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

UNION

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

U

Wallace trabalha
nos projetos 20 e
30.

COD_PROJ	NOME
10	Automatizacao
20	Reorganizacao
30	Novos Beneficios

UNION ALL

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

UNION ALL

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

UNION ALL

Wallace trabalha
nos projetos 20 e
30.

COD_PROJ	NOME
10	Automatizacao
30	Novos Beneficios
20	Reorganizacao
30	Novos Beneficios

INTERSECT

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Wallace tanto como gerente do departamento onde o projeto é realizado quanto como trabalhador do projeto

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

INTERSECT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

INTERSECT

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

INTERSECT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

∩

Wallace trabalha
nos projetos 20 e
30.

⋮ COD_PROJ	NOME
30	Novos Benefícios

EXCEPT

Faça uma lista de todos os códigos de projeto que envolvem um empregado cujo sobrenome é Wallace como gerente do departamento onde o projeto é realizado, mas não como trabalhador do projeto

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

EXCEPT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

EXCEPT

```
SELECT p.COD_PROJ, p.NOME
FROM PROJETO p, DEPARTAMENTO d, EMPREGADO e
WHERE p.COD_DEPTO = d.COD_DEPTO
AND e.CPF = d.CPF_GERENTE AND SOBRENOME = 'Wallace'
```

EXCEPT

```
SELECT p.COD_PROJ, p.Nome
FROM PROJETO p, TRABALHA_EM t, EMPREGADO e
WHERE p.COD_PROJ = t.COD_PROJ
AND e.CPF = t.CPF AND SOBRENOME = 'Wallace'
```

Wallace gerencia
DEPTO 4 que é
responsável pelos
projetos 10 e 30.

-

Wallace trabalha
nos projetos 20 e
30.

⌵ COD_PROJ	NOME
10	Automatizacao

Funções de Agregação

Funções de Agregação

São usadas para **resumir informações** de várias tuplas em uma síntese de tupla única

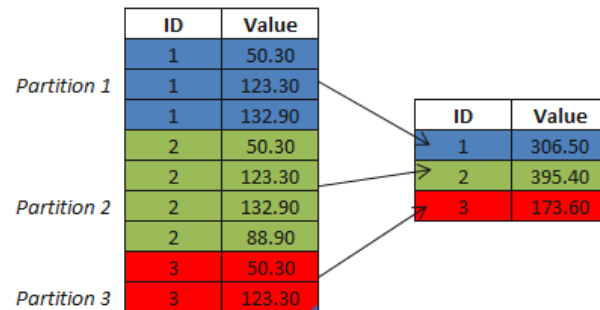
Agrupamento

- Cria subgrupos de tuplas antes do resumo

Funções de agregação

- **COUNT**, **SUM**, **MAX**, **MIN** e **AVG**

Essas funções podem ser usadas na cláusula **SELECT** ou em uma cláusula **HAVING**



	ID	Value
Partition 1	1	50.30
	1	123.30
	1	132.90
Partition 2	2	50.30
	2	123.30
	2	132.90
	2	88.90
Partition 3	3	50.30
	3	123.30

ID	Value
1	306.50
2	395.40
3	173.60

Funções de Agregação

Selecionar a soma dos salários de todos os empregados, o salário máximo, o salário mínimo e o valor médio dos salários

```
SELECT SUM(SALARIO) , MAX(SALARIO) , MIN(SALARIO) , AVG(SALARIO)
FROM EMPREGADO;
```

SUM(SALARIO)	MAX(SALARIO)	MIN(SALARIO)	AVG(SALARIO)
281000	55000	25000	35125

Funções de Agregação

Selecionar a média dos salários de todos os empregados do departamento de Pesquisa

```
SELECT AVG(SALARIO)
FROM (EMPREGADO e JOIN DEPARTAMENTO d
      ON e.COD_DEPTO = d.COD_DEPTO)
WHERE d.NOME = 'Pesquisa';
```

: AVG(SALARIO)

33250

Funções de Agregação

Selecionar o número total de empregados da empresa

```
SELECT COUNT ( * )
FROM EMPREGADO;
```

EMPREGADO

Nome	INICIA...	SOBR...	CPF	DNAS...	ENDE...	SEXO	SALA...	CPF_S...	COD_DEPTO
John	B	Smith	12345...	1965-01...	731 Fon...	M	30000	33344...	5
Franklin	T	Wong	33344...	1955-12...	638 Vos...	M	40000	88866...	5
Joyce	A	English	45345...	1972-07...	5631 Ri...	F	25000	33344...	5
Ramesh	K	Narayan	66688...	1962-09...	975 Fire...	M	38000	33344...	5
James	E	Borg	88866...	1937-11...	450 Sto...	M	55000	NULL	1
Jennifer	S	Wallace	98765...	1941-06...	291 Ber...	F	43000	88866...	4
Ahmad	V	Jabbar	98798...	1969-03...	980 Dall...	M	25000	98765...	4
Alicia	J	Zelaya	99988...	1968-01...	3321 C...	F	25000	98765...	4



⋮ COUNT(*)

8

Funções de Agregação

Selecionar o número total de empregados do departamento de Pesquisa

```
SELECT COUNT (*)  
FROM EMPREGADO e, DEPARTAMENTO d  
WHERE e.COD_DEPTO = d.COD_DEPTO  
AND d.NOME = 'Pesquisa';
```

⋮ COUNT(*)

4

Funções de Agregação

Selecionar o número de salários distintos da empresa

```
SELECT COUNT(DISTINCT SALARIO)
FROM EMPREGADO;
```

⋮ COUNT(DISTINCT SALARIO)

6

Funções de Agregação

Selecionar os empregados que possuem 2 ou mais dependentes

```
SELECT NOME, SOBRENOME
FROM EMPREGADO e
WHERE (SELECT COUNT (*)
       FROM DEPENDENTE d
       WHERE e.CPF = d.EMP_CPF) >= 2;
```

NOME	SOBRENOME
Franklin	Wong
John	Smith

GROUP BY

Particionar a relação em subconjunto de tuplas baseado no atributo(s) de agrupamento

- Aplicar a função de agregação a cada grupo desse tipo independentemente

Cláusula **GROUP BY**

- Especifica os atributos de agrupamento

Se houver NULLs no atributo de agrupamento

- Um grupo separado é criado para todas as tuplas com um valor NULL no atributo de agrupamento.

GROUP BY

Para cada departamento, selecionar o código do departamento, o número de empregados que trabalham no departamento, e o salário médio deles

```
SELECT COD_DEPTO, COUNT(*), AVG(SALARIO)
FROM EMPREGADO
GROUP BY COD_DEPTO
```

Vamos ver mais detalhes no próximo slide...

GROUP BY

```
SELECT COD_DEPTO, COUNT(*), AVG(SALARIO)
FROM EMPREGADO
GROUP BY COD_DEPTO
```

NOME	...	CPF	...	SALARIO	CPF_SUPERVISOR	COD_DEPTO
João		12345678966		30.000	33344555587	5
Fernando		33344555587		40.000	88866555576	5
Ronaldo		66688444476		38.000	33344555587	5
Joice		45345345376		25.000	33344555587	5
Alice		99988777767		25.000	98765432168	4
Jennifer		98765432168		43.000	88866555576	4
André		98798798733		25.000	98765432168	4
Jorge		88866555576		55.000	NULL	1

Resposta da Consulta

COD DEPTO	Count (*)	Avg (Salario)
5	4	33.250
4	3	31.000
1	1	55.000

Tabela EMPREGADO

GROUP BY

Para cada projeto, selecione o código do projeto, o nome do projeto, e o número de empregados que trabalham no projeto

```
SELECT p.COD_PROJ, NOME, COUNT (*)  
FROM PROJETO p, TRABALHA_EM t  
WHERE p.COD_PROJ = t.COD_PROJ  
GROUP BY p.COD_PROJ, NOME;
```

IMPORTANTE: todos os atributos que constam do SELECT e que não são agregações precisam aparecer no GROUP BY

O SQLite não obriga, mas a consulta fica incorreta sem mensagem de erro

HAVING

A cláusula HAVING especifica uma condição que deve ser satisfeita pelo agrupamento

Em geral é usada junto com GROUP BY

OBS: O SQLite aceita HAVING sem GROUP BY, tratando a tabela toda como um único grupo

HAVING

Para cada projeto no qual trabalham mais de 2 empregados, selecione o código do projeto, o nome do projeto, e o número de empregados que trabalham no projeto

```
SELECT p.COD_PROJ, NOME, COUNT (*)  
FROM PROJETO p, TRABALHA_EM t  
WHERE p.COD_PROJ = t.COD_PROJ  
GROUP BY p.COD_PROJ, NOME  
HAVING COUNT (*) > 2;
```

HAVING

COD_PROJ	NOME	...	CPF	COD_PROJ	HORAS
1	ProdutoX		12345678966	1	32,5
1	ProdutoX		45345345376	1	20,0
2	ProdutoY		12345678966	2	7,5
2	ProdutoY		45345345376	2	20,0
2	ProdutoY		33344555587	2	10,0
3	ProdutoZ		66688444476	3	40,0
3	ProdutoZ		33344555587	3	10,0
10	Informatização		33344555587	10	10,0
10	Informatização		99988777767	10	10,0
10	Informatização		98798798733	10	35,0
20	Reorganização		33344555587	20	10,0
20	Reorganização		98765432168	20	15,0
20	Reorganização		88866555576	20	NULL
30	Novos Benefícios		98798798733	30	5,0
30	Novos Benefícios		98765432168	30	20,0
30	Novos Benefícios		99988777767	30	30,0

```
SELECT p.COD_PROJ, NOME, COUNT(*)  
FROM PROJETO p, TRABALHA_EM t  
WHERE p.COD_PROJ = t.COD_PROJ  
GROUP BY p.COD_PROJ, NOME  
HAVING COUNT(*) > 2;
```

Esses grupos **não satisfazem**
à condição da cláusula HAVING

Resultado da consulta após a cláusula a aplicação da cláusula WHERE mas
ANTES da aplicação da cláusula HAVING

HAVING

```
SELECT p.COD_PROJ, NOME, COUNT(*)  
FROM PROJETO p, TRABALHA_EM t  
WHERE p.COD_PROJ = t.COD_PROJ  
GROUP BY p.COD_PROJ, NOME  
HAVING COUNT(*) > 2;
```

COD_PROJ	NOME	...	CPF	COD_PROJ	HORAS
1	ProdutoX		12345678966	1	32,5
1	ProdutoX		45345345376	1	20,0
2	ProdutoY		12345678966	2	7,5
2	ProdutoY		45345345376	2	20,0
2	ProdutoY		33344555587	2	10,0
3	ProdutoZ		66688444476	3	40,0
3	ProdutoZ		33344555587	3	10,0
10	Informatização		33344555587	10	10,0
10	Informatização		99988777767	10	10,0
10	Informatização		98798798733	10	35,0
20	Reorganização		33344555587	20	10,0
20	Reorganização		98765432168	20	15,0
20	Reorganização		88866555576	20	NULL
30	Novos Benefícios		98798798733	30	5,0
30	Novos Benefícios		98765432168	30	20,0
30	Novos Benefícios		99988777767	30	30,0

Resultado da consulta

COD_PROJ	NOME	COUNT(*)
2	ProdutoY	3
10	Informatização	3
20	Reorganização	3
30	Novos Benefícios	3

SELECT – Resumo Final

```
SELECT <lista atributo e função>  
FROM <lista tabela>  
[ WHERE <condição> ]  
[ GROUP BY <atributo(s) de agrupamento> ]  
[ HAVING <condição de grupo> ]  
[ ORDER BY <lista atributos> ];
```

Para praticar...

Vamos usar o SQLite

No Google Classroom, baixe o arquivo airport.db e salve numa pasta do seu computador

No VSCode, abra essa pasta pelo menu File / Open Folder

Clique no SQLite Explorer e depois em New Query

Escreva a consulta, clique com o botão da direita e escolha Run Query

O passo a passo completo está no material **SQLite no VSCode**

Atenção: no SQLite a comparação com = diferencia maiúsculas de minúsculas ('ES' não é igual a 'es')

SQLite Online

Se não conseguir instalar a extensão no VSCode, você também pode usar o <https://sqliteonline.com/>



```

aircrafttype (AircraftTypeID, AircraftName)
airport (AirportID, AirportCode, AirportName, CityName,
          CountryCode, NumRunways, NumTerminals)
aircraft (AircraftID, AircraftTypeID, RegNum, LastMaintEnd,
          NextMaintBegin, NextMaintEnd);
          AircraftTypeID referencia aircrafttype
route (RouteID, Origin, Destination, Distance, Duration, RouteStatus)
          Origin references airport (AirportID)
          Destination references airport (AirportID)
class (ClassID, ClassName)
flight (FlightID, RouteID, AircraftID)
          AircraftID referencia aircraft
          RouteID referencia route
flightclass (FlightID, ClassID, MaxSeats, BasePrice)
          FlightID referencia flight
          ClassID referencia class
flightdep (FlightID, DepDay, DepTime); //horários de vôos
          FlightID referencia flight
log (RecordID, ByUser, Note, EventTime)
pax (RecordID, FlightID, FlightDate, ClassID, PaxName, PaxRef, Note)
          FlightID referencia flight
          ClassID referencia class
stats (FlightID, FlightDate, ClassID, CurrSeats, CurrPrice)
          FlightID referencia flight
          ClassID referencia class

```

Exercícios

1. Faça uma consulta que retorna as rotas (RouteID, Origin, Destination) que possuem como origem o aeroporto LHR. Use o comando de JOIN apropriado.
2. Repita a consulta anterior, agora usando subconsulta para produzir a resposta.
3. Faça uma consulta que retorna o código dos aeroportos de origem e destino de cada rota – a consulta deve retornar o código da rota, o código do aeroporto origem (renomeado para Origem) e o código do aeroporto destino (coluna AirportCode) renomeado como Destino. O retorno deve estar ordenado pelo código do aeroporto de origem e logo após, pelo código do aeroporto de destino. Use o comando de JOIN apropriado.
4. Faça uma consulta que retorne todas as informações de voo (flight) e os horários de partida desse voo (tabela flightdep). Use o operador de junção apropriado. O resultado deve estar ordenado por FlightID.
5. Faça uma consulta que retorna os voos (FlightID) e os IDs das classes que esse voo opera. Caso um voo não tenha registro de classe, mesmo assim ele deve aparecer no resultado.

Exercícios

6. Faça uma consulta que retorna os dados dos vôos para os quais existe alguma reserva (tabela pax). Use subconsulta para resolver essa questão.
7. Faça uma consulta que retorna o número do vôo e o nome do passageiro que reservou o vôo (tabela pax). O número do vôo deve aparecer no resultado mesmo se não houver nenhuma reserva para ele. A resposta deve estar ordenada pelo número do vôo e pelo nome do passageiro.

Referência

Sistemas de Banco de Dados. Ramez E. Elmasri, e Shamkant Navathe, 2005, 4a edição, Ed. Pearson - Addison Wesley.

Referência

Tutorial SQLite:

<https://www.sqlitetutorial.net/>

Sintaxe: <https://www.sqlite.org/lang.html>



Referência

W3 Schools (apresenta sintaxe para vários BDs diferentes)

- <https://www.w3schools.com/sql/default.asp>



Agradecimento

Slides baseados no material de Daniel Oliveira

Perguntas?

Vanessa Braganholo

vanessa@ic.uff.br